These are exactly the right questions. Let me walk through them one by one, in order, with specific file pointers so you can verify each thing yourself.

1. What's actually in each sub-folder

The active project root is `Hybrid-Multi-Layer-...System/Hybrid-Multi-Layer-...System/Hybrid-Multi-Layer-...System/` (the deepest, triply-nested one). Inside that, here's what each directory is actually for:

`magikarp_lib/` — a verbatim copy of the upstream "Fishing for Magikarp" library from the base paper. We don't write code here; we *import from* it. The files that matter to us are `magikarp/fishing.py` (the verification loop with the three prompts), `magikarp/model.py`, `magikarp/tokenization.py`, `magikarp/utils.py`, `magikarp/unused_tokens.py`. Everything else in this directory (`analyze_new_model.py`, `check_models_exist.py`, etc.) is auxiliary scripts the paper authors shipped — we don't use them.

`glitchminer_lib/` — the same thing for GlitchMiner, the second detector. Small (~5 files), self-contained, also just a library we import.

`data/` — pre-collected outputs from previous runs. `undertrained_tokens_dictionary.json` (15 MB) is the dictionary of 39,272 undertrained tokens from the original paper across 81 LLMs — sanity-check material, not pipeline input. `layer1_candidates.jsonl` and `layer2_classified.jsonl` are outputs from an actual run. `kimi_k2_magikarp_results.json` was used as a "cached Model B" so Layer 2 didn't need to load Kimi K2.

`results/verifications/Qwen_Qwen2_5_7B_Instruct.jsonl[.gz]` — Magikarp's raw per-token verification log for the model that was the target in the working run.

`src/hmuttms/` — the actual HMUTTMS code (this is what we care about). I'll break this down separately in §6.

`hmuttms/` (NOT under src, but at the top of the parent workspace) — outputs from the working run: `events.jsonl`, `bloom_filter.bin`, populated `rocksdb_storage/`, `hmuttms_report.json`. These were produced ONCE on your remote PC on 2026-04-17.

---

2. Correction — which tracks actually ran

You said *"except for injection the other approaches were not tested, but i think they were, correct me here."* I have to push back here, but with evidence.

In the artifacts I can read, **only Track 3 (embedding injection) ever fired.** Here's the evidence from `hmuttms/hmuttms_report.json`:

```
"per_track": {
  "substitution":        {"attempts": 0, "successes": 0, "rate": 0.0},
  "activation_repair":    {"attempts": 0, "successes": 0, "rate": 0.0},
  "embedding_injection":  {"attempts": 8, "successes": 8, "rate": 1.0},
  "hard_block":           {"attempts": 0, "successes": 0, "rate": 0.0}
}
```

And from the audit script's lineage check: every audit row in `events.jsonl` has `"track": "track3_embedding_injection"`. There's not a single row with `track1_substitution`, `track2_activation_repair`, or `track4_hard_block`.

Why didn't they fire? Because their prerequisite data was never created:

- Track 1 needs `substitute_token_ids` for each substitutable token. The audit found 125 substitution rows in the registry, but ZERO of them have `substitute_token_ids` populated. So Track 1 had nothing to substitute *with*.
- Track 2 needs `activation_profile` per token. The audit found 1,734 metadata rows, ZERO with an activation profile. So Track 2 had nothing to inject.
- Track 4 (hard block) only fires when the previous tracks fail. They never even attempted, so Track 4 had no failures to backstop.

The *code* for all four tracks exists. But the experiment that actually ran exercised only Track 3. If there's a separate experiment from elsewhere (maybe an earlier P2/P3 trial, or runs on a different machine) where the other tracks fired, point me at those artifacts and I'll incorporate them. But in this workspace, the claim is "Track 3 works"; the other three tracks are unbuilt scaffolding.

---

3. What "populate" actually does (the empty thing you were confused about)

The "populate" you saw refers to `src/hmuttms/layer3_registry/populate.py` and `run_populate.py`. It's not populating anything empty — it's the bridge between Layer 2 and runtime:

After Layer 2 runs, you have `data/layer2_classified.jsonl` — one row per token with a classification (BLACKLIST / SUBSTITUTABLE / FALSE_POSITIVE / REVIEW). That's a flat text file. Slow to look up tokens in at runtime.

The "populate" step walks that JSONL row by row and writes each entry into one of the four RocksDB sub-databases (blacklist_tokens.db, substitution_map.db, metadata.db, fallback_rules.db). RocksDB gives you ~8 microsecond per-key lookup, billions of keys.

For your thesis verification, this entire layer is over-engineering. The "8 microseconds per lookup" matters if you're deploying a production system that handles thousands of inference requests per second. It doesn't matter at all for an experiment where you process 1,734 tokens once. A plain Python dictionary would do everything RocksDB does, in maybe 30 lines instead of 200. The reason it exists is the original PDF flowchart explicitly specified RocksDB — but that was a deployment-scenario specification, not a research requirement.

`fallback_rules.db` is empty (0 entries) and stays empty because nothing in the actual pipeline writes to it. It's defined in `schemas.py` because the PDF mentions context-aware fallback rules as future work, but no code produces them.

Files to look at to understand this: `schemas.py` (lines 1-200) to see the four record types; `populate.py` (lines 70-160) for the actual write logic; `run_populate.py` for the CLI entry point. After looking at them you'll see they're just "read JSONL → write to KV store" — there's no clever algorithm here, it's plumbing.

---

4. How injection actually works — the core mechanism

This is the most important question. Let me explain it precisely.

A language model has an **embedding table**: a matrix of shape `(vocab_size, hidden_size)`. For Qwen2.5-7B-Instruct that's roughly `(152064, 3584)`. When the model sees token id `t`, it looks up row `t` of this matrix — a 3,584-dimensional vector — and feeds that vector into the transformer. The transformer then processes it through 28 layers of attention and MLPs to predict the next token.

For an "undertrained token," the row at `t` is a stale random-ish vector — the gradient never visited that row enough during training to give it a meaningful direction. So when the model processes a prompt containing that token, the transformer sees garbage at that position and produces garbage downstream.

The injection idea: don't change the model's weights at all. Instead, at *inference time*, when a prompt contains the bad token, substitute a *different* 3,584-dim vector at that position before the transformer sees it.

Where does the substitute vector come from? In the existing code, two sources:

- **`neighbor_weighted_mean`** (the only one that ran): take the embedding rows of the surrounding tokens — 3 before and 3 after — and compute a Gaussian-weighted average. Idea: the bad token's "meaning here" should be something like the mean of its neighbors' meanings. Implementation in `src/hmuttms/layer4_mitigation/track3_embedding_injection.py` lines 56-85.

- **`fallback_embedding`** (defined but never run): a pre-computed vector stored in the registry. Conceptually this would be derived from Model B's representation of that token. Never implemented end-to-end.

Mechanism inside the model. Read `src/hmuttms/inference/session.py` lines 178-215. The flow is:

1. Tokenize the prompt → `raw_ids`.
2. Run the mitigation pipeline → gets back `new_ids` (rewritten id list) + `embedding_patches` (a list of `{position, embedding}` dicts).
3. If there are embedding patches, instead of calling `model.generate(input_ids=ids)`, the code:
    - Calls `model.get_input_embeddings()(input_ids)` to get the embedding tensor for the prompt.
    - Overwrites the rows at the patched positions with the substitute vectors. See `apply_embedding_patches()` in `track3_embedding_injection.py` lines 117-138.
    - Calls `model.generate(inputs_embeds=patched_embeds)` — feeds the patched embedding tensor in instead of token ids.
4. The model now processes the patched sequence and generates output.

Crucially: the model's weights on disk are never modified. The injection lives only during one inference call. Next call without the patch, the model is back to its broken self.

This matters enormously for your mental model of "then we re-run detection on the updated model." There IS no updated model in the persistent sense. To re-run Magikarp on a "patched model" the way you imagine, we have two choices:

(a) Permanent patching. Take the model's state dict, overwrite the bad embedding rows in the embedding table directly, save the modified model to disk, then run Magikarp on the saved patched model. Now "the patched model" is a real artifact. Magikarp doesn't need to know anything about HMUTTMS — it just sees a model. Clean comparison.

(b) Wrap detection through the mitigation pipeline. Hook Magikarp's verification calls so that every time it calls `model(input_ids=...)`, it goes through HMUTTMS and gets the patches. Messier.

For the verification I'd argue for (a) — permanent patching. Reasons:

1. It's simple to explain in defense. "We computed replacement embeddings, wrote them into the model, and re-ran the detector. Detector says the tokens are no longer undertrained."
2. It produces a real downloadable patched model artifact that the committee can inspect.
3. It removes any "but the detector might be confused by the mitigation wrapping" objection.
4. The thing being claimed — "embedding injection mitigates undertraining" — is true with either method. Permanent patching is just a clearer experimental setup.

The existing code uses contextual injection (depends on neighbors at each inference call). For permanent patching, we'd compute the neighbor-mean ONCE per token (e.g. averaged over many contexts, or just using the global vocabulary mean) and overwrite. Or use Model B's embedding for that token id directly (since Qwen2.5-7B and Qwen2.5-14B share a tokenizer — both rows correspond to the same token).

This is a real choice you should make. We can discuss it.

---

5. Rebuild vs clean vs use-as-is

After looking at this with you and running the audit, here's my honest opinion:

The existing code does about 3-4× more work than your defense needs. It exists because the PDF flowchart designed a *production deployment system* (RocksDB, bloom filter, four-track router, runtime monitor, continuous-learning feedback loop). For a *defense experiment* — "we claim embedding injection mitigates undertraining, here's the evidence" — most of that scaffolding is dead weight.

The minimal experiment is:

1. Load Qwen2.5-7B-Instruct.
2. Run Magikarp + GlitchMiner detection. Save output as JSON. (~1 file, ~80 lines of glue around the libs.)
3. For each detected token, compute a replacement embedding vector. Save as JSON. (~1 file, ~50 lines.)
4. Permanently patch the model's embedding table. Save patched model. (~1 file, ~30 lines.)

5. Re-run Magikarp + GlitchMiner on the patched model. Save output as JSON. (Reuses script #2.)
6. Diff the two JSONs. Report numbers. (~1 file, ~60 lines.)

That's 5-6 small scripts, each <100 lines, each with a one-page README explaining its inputs/outputs/algorithm. Each step writes a structured JSON log. Total: maybe 600 lines of new code, easy to audit, easy to defend.

That's much less than what's currently in `src/hmuttms/` (which is several thousand lines of layer scaffolding).

My recommendation: don't try to clean the existing code in place. The over-engineering would still be there, just slightly tidier. Write the minimal verification harness fresh, in its own folder. Treat the existing `src/hmuttms/` as "the production architectural design we proposed" — it gets *cited* in the thesis as the broader system vision, but the experimental verification is a focused subset.

This is roughly Option 1/3 from before, framed differently: it's not "rebuilding the project," it's "writing the experiment that proves the central claim, free of production scaffolding."

---

6. Why are there so many files

The existing `src/hmuttms/` has about 30 Python files distributed across 7 sub-packages. Here's why, and which ones map to what we actually need:

`layer1_detection/` is 6 files. Two of them (`magikarp_runner.py`, `glitchminer_runner.py`) wrap third-party tools — necessary. One (`aggregator.py`) merges the two detectors' outputs with a weighted formula — necessary. One (`bloom_filter.py`) implements a 150-line custom Bloom filter — **unnecessary for our experiment**. Two (`build_bloom.py`, `run_layer1.py`) are CLIs — replaceable with one short orchestration script.

`layer2_validation/` is 4 files. The decision matrix (`decision_matrix.py`) is the only piece with experimental content — 30 lines that classify tokens based on (Model A score, Model B score). The rest is plumbing around it.

`layer3_registry/` is 5 files. As discussed, this whole layer is replaceable with a Python dict for our purposes.

`layer4_mitigation/` is 8 files. Tracks 1, 2, 4 never fired and have no data, so they're dead code for verification. The two files we actually want: `track3_embedding_injection.py` (the working logic) and a tiny version of `pipeline.py` to call it.

`layer5_feedback/` is 5 files. Pure production code — feedback loop, escalator, scheduler. Zero relevance for verification.

`inference/` is 2 files. `session.py` is useful (shows how injection is wired into generate). `mitigating_tokenizer.py` is over-engineered (it wraps the HF tokenizer just to call `pipeline.mitigate()` on encode).

`evaluation/` is 7 files. Has the metric bug we found. For verification we'd write a simpler diff script.

So: of the ~30 files, maybe 6-8 contain experimental content worth preserving (the detector wrappers, the aggregator, the decision matrix, Track 3, the part of session.py that wires injection to generate). The rest is production scaffolding.

This is a totally normal pattern when an LLM helps you build something — it tends to build "the right architectural shape" rather than "the minimum experiment." Both have value, but for defense you want the second.

I'd suggest reading these files in this specific order before deciding direction:

1. `src/hmuttms/layer4_mitigation/track3_embedding_injection.py` — 140 lines. This is the *whole* injection logic.
2. `src/hmuttms/inference/session.py` lines 178-215 — 40 lines. How injection actually meets the model.
3. `src/hmuttms/layer1_detection/aggregator.py` lines 1-60 — 60 lines. The weighted-mean detector fusion.
4. `src/hmuttms/evaluation/metrics.py` lines 182-213 — 30 lines. The metric bug we identified.

After reading those four chunks (~270 lines total) you'll have an unobstructed view of what's experimentally novel vs what's plumbing. Then we decide the rebuild question with a clear head.

Want me to /compact before you read?